Travaux Pratiques 1–2

Premier contact avec une infrastructure Cloud locale

VirtualBox + Ubuntu Server + SSH + Vagrant

Pr. A. ABASSI

Faculté des Sciences et Techniques (FST) de Settat

Université Hassan 1er, Settat — Maroc

Continuité pédagogique

Ce TP constitue une première mise en pratique des concepts fondamentaux du Cloud Computing étudiés en cours.
Contrairement aux plateformes Cloud publiques telles que Google Cloud, Microsoft Azure ou Amazon Web Services, ce TP fonctionne entièrement **hors ligne** sur le poste de travail de l'étudiant.
L'environnement utilisé repose sur :

- **VirtualBox** : hyperviseur permettant d'exécuter des machines virtuelles ;
- **Ubuntu Trusty 14.04** : système d'exploitation de la machine virtuelle fourni sous forme de box Vagrant ;
- **Terminal/PowerShell de l'OS Hôte** : client SSH de la machine physique ;
- **Vagrant** : outil permettant d'automatiser le provisionnement et la gestion de machines virtuelles.

L'objectif n'est pas de reproduire complètement un Cloud public, mais d'illustrer localement plusieurs de ses concepts fondamentaux :
IaaS, virtualisation, réseau virtuel, SSH, sécurité, utilisateurs, permissions et Infrastructure as Code.

1 Objectifs pédagogiques

À la fin de ce TP, l'étudiant devra être capable de :

- expliquer le principe d'une machine virtuelle et son rôle dans le modèle IaaS ;
- identifier les ressources virtuelles attribuées à une VM : CPU, mémoire, disque et réseau ;
- utiliser une box Vagrant locale ;
- démarrer une machine virtuelle Ubuntu avec Vagrant et VirtualBox ;
- identifier le nom d'utilisateur et l'adresse IP d'une VM ;
- accéder à la machine virtuelle en **SSH à partir du système d'exploitation hôte** ;

— vérifier le fonctionnement du serveur SSH ;
— configurer un pare-feu Linux avec **UFW** ;
— créer des utilisateurs et des groupes Linux afin d'illustrer le principe du **moindre privilège** ;
— utiliser Vagrant pour automatiser le déploiement d'une VM ;
— comprendre la différence entre une connexion SSH classique et la commande `vagrant ssh` ;
— faire le lien entre une infrastructure Cloud locale et une plateforme Cloud publique telle que GCP.

2 Préparation et environnement

2.1 Environnement logiciel

Le TP est conçu pour fonctionner **hors ligne** après préparation de l'environnement. Chaque poste doit disposer des éléments suivants :

— **Oracle VirtualBox** installé ;
— **Vagrant** installé ;
— la box Vagrant **ubuntu/trusty64** fournie par l'enseignant ;
— un terminal sur la machine hôte : PowerShell / Invite de commandes sous Windows ou Terminal sous macOS/Linux ;
— au minimum :
  — 4 Go de RAM disponibles ;
  — 20 Go d'espace disque disponible ;
  — virtualisation matérielle activée dans le BIOS/UEFI.

Important — Box fournie par l'enseignant

L'enseignant fournit aux étudiants le fichier :

`ubuntu-trusty64.box`

Cette box contient une machine Ubuntu Trusty 14.04 préparée pour être utilisée avec Vagrant et VirtualBox.
Aucun téléchargement de la box depuis Internet n'est nécessaire.
L'étudiant doit simplement ajouter la box locale à Vagrant.

2.2 Installation de Vagrant

Avant de commencer l'étape 4, Vagrant doit être installé sur la machine hôte.

1. Installer Vagrant à partir de l'installateur fourni par l'enseignant ou depuis le site officiel si une connexion Internet est disponible.
2. Vérifier l'installation depuis PowerShell ou le Terminal :

```
vagrant --version
```

3. Une version de Vagrant doit être affichée, par exemple :

```
Vagrant 2.4.x
```

2.3 Installation de la box locale

Supposons que l'enseignant ait fourni le fichier :

```
ubuntu-trusty64.box
```

Placez ce fichier dans un dossier facilement accessible, par exemple :

```
C:\FST\TP-Cloud\
```

Sous PowerShell :

```
cd C:\FST\TP-Cloud
```

Puis ajoutez la box à Vagrant :

```
vagrant box add ubuntu/trusty64 ubuntu-trusty64.box
```

Si l'opération est réussie, Vagrant affiche notamment le provider :

```
Provider: virtualbox
```

Vérifier ensuite que la box est disponible :

```
vagrant box list
```

Vous devez obtenir une ligne similaire à :

```
ubuntu/trusty64 (virtualbox, ...)
```

Validation

À partir de ce moment, Vagrant possède localement la box `ubuntu/trusty64`. L'étudiant n'a plus besoin de télécharger cette box depuis Internet.

3 Correspondance avec les services Cloud

Cloud public	Environnement local	Concept
Compute Engine	VirtualBox VM	IaaS
VPC	VirtualBox Network	Réseau virtuel
Firewall VPC	UFW	Sécurité réseau
SSH	OpenSSH / Terminal Hôte	Administration distante
IAM	Utilisateurs/Groupes Linux	Gestion des droits
Provisionnement Cloud	Vagrant	Automatisation
Cloud Console	Terminal OS Hôte	Administration

Question de réflexion

Pourquoi peut-on considérer une machine virtuelle VirtualBox comme une illustration simplifiée du modèle IaaS ?

4 Instructions de manipulation

Étape 1 — Découverte de la VM et du modèle IaaS (25 min)

Cette étape permet d'observer une machine virtuelle et d'identifier les ressources qui lui sont attribuées.

1.1 Création de la VM avec Vagrant

Contrairement à une création manuelle dans VirtualBox, nous allons utiliser Vagrant pour créer automatiquement la machine virtuelle.

Créer un dossier de travail :

```
mkdir tp-cloud-vagrant
cd tp-cloud-vagrant
```

Initialiser le projet avec la box locale :

```
vagrant init ubuntu/trusty64
```

Cette commande crée un fichier :

```
Vagrantfile
```

Attention

La commande `vagrant init` ne crée pas encore la machine virtuelle.
Elle crée uniquement le fichier `Vagrantfile`.

1.2 Configuration du Vagrantfile

Ouvrir le fichier `Vagrantfile` et remplacer son contenu par :

```
Vagrant.configure("2") do |config|

config.vm.box = "ubuntu/trusty64"

config.vm.hostname = "vm-tp1-cloud"

config.vm.network "private_network",
ip: "192.168.56.110"

config.vm.provider "virtualbox" do |vb|

vb.name = "vm-tp1-cloud"

vb.memory = 1024

vb.cpus = 1

end

end
```

Cette configuration indique notamment :

— utilisation de la box `ubuntu/trusty64` ;
— nom de la machine : `vm-tp1-cloud` ;
— adresse IP privée : `192.168.56.110` ;
— mémoire : 1024 Mo ;
— processeur : 1 CPU.

1.3 Démarrage de la VM

Exécuter :

```
vagrant up
```

Vagrant va :

1. rechercher la box `ubuntu/trusty64` localement ;
2. créer une machine VirtualBox ;
3. configurer la mémoire et le CPU ;
4. configurer le réseau ;
5. démarrer la machine ;
6. configurer l'accès SSH.

Important — Fonctionnement hors ligne

Comme la box `ubuntu/trusty64` a déjà été ajoutée localement, Vagrant n'a pas besoin de télécharger la box depuis Internet.

1.4 Vérifier l'état de la VM

Exécuter :

```
vagrant status
```

La machine doit apparaître comme :

```
default running (virtualbox)
```

Vous pouvez également vérifier la machine directement dans VirtualBox.

1.5 Connexion à Ubuntu avec Vagrant

Exécuter :

```
vagrant ssh
```

Si la connexion fonctionne, l'invite de commande ressemble à :

```
vagrant@vm-tp1-cloud:~$
```

Vérifier l'utilisateur :

```
whoami
```

Résultat attendu :

```
vagrant
```

Vérifier le nom de la machine :

```
hostname
```

Résultat attendu :

```
vm-tp1-cloud
```

1.6 Identifier les ressources virtuelles

Exécuter :

```
uname -a
```

Puis :

```
lscpu
```

Puis :

```
free -h
```

Enfin :

```
df -h
```

Compléter le tableau :

Ressource	Valeur observée	Valeur configurée
CPU		1
RAM		1024 MB
Disque		
Réseau		192.168.56.110

Étape 2 — Réseau virtuel et accès SSH (30 min)

L'objectif de cette étape est de comprendre comment un système d'exploitation hôte peut administrer une VM Linux à distance.

2.1 Identifier l'adresse IP de la VM

Depuis la VM :

```
hostname -I
```

Vous devriez observer notamment :

```
192.168.56.110
```

Vous pouvez également utiliser :

```
ip addr show
```

Repérer l'interface correspondant au réseau privé.
Compléter :

USER = _____

IP_VM = _____

2.2 Vérifier le serveur SSH

Ubuntu Trusty utilise encore l'architecture de démarrage **Upstart**. La commande `systemctl` n'est donc pas utilisée dans cette version.

Vérifier l'état du service SSH :

```
sudo service ssh status
```

Si nécessaire, démarrer le service :

```
sudo service ssh start
```

Vérifier ensuite :

```
sudo service ssh status
```

Vérifier que le serveur écoute sur le port 22 :

```
sudo netstat -tlnp | grep :22
```

Si `netstat` n'est pas disponible :

```
sudo apt-get install -y net-tools
```

Puis refaire :

```
sudo netstat -tlnp | grep :22
```

2.3 Tester SSH depuis l'OS Hôte

Quitter la session SSH :

```
exit
```

Depuis PowerShell sous Windows, tester :

```
ssh vagrant@192.168.56.110
```

Attention — Connexion SSH classique

Cette commande utilise une authentification SSH classique.
Elle est différente de :

```
vagrant ssh
```

La commande `vagrant ssh` utilise automatiquement la configuration SSH et la clé privée gérées par Vagrant.

Pour cette partie, l'objectif pédagogique est de comprendre le principe :

OS Hôte ⟶ réseau virtuel ⟶ port 22 ⟶ serveur SSH

2.4 Vérification du réseau depuis Windows

Depuis PowerShell, tester la connectivité :

```
ping 192.168.56.110
```

Puis tester le port SSH :

```
Test-NetConnection 192.168.56.110 -Port 22
```

Le résultat doit indiquer :

```
TcpTestSucceeded : True
```

Validation SSH

Si :

```
TcpTestSucceeded : True
```

le port SSH de la VM est accessible depuis le système d'exploitation hôte.

Étape 3 — Sécurité et Firewall UFW (25 min)

Cette étape permet d'illustrer localement le principe des règles de pare-feu utilisées dans les infrastructures Cloud.

ATTENTION — Ordre critique

Toujours autoriser le flux SSH avant d'activer le pare-feu.
Sinon, une connexion SSH distante peut être bloquée.

3.1 Vérifier UFW

Depuis la session SSH :

```
sudo ufw status
```

Si UFW n'est pas installé :

```
sudo apt-get update
sudo apt-get install -y ufw
```

3.2 Autoriser SSH

Autoriser le port SSH :

```
sudo ufw allow 22/tcp
```

Vérifier la règle :

```
sudo ufw status numbered
```

3.3 Activer UFW

Activer le pare-feu :

```
sudo ufw enable
```

Puis :

```
sudo ufw status verbose
```

La règle SSH doit apparaître.

3.4 Bloquer un port d'application

Ajouter une règle refusant le trafic TCP sur le port 8080 :

```
sudo ufw deny 8080/tcp
```

Puis :

```
sudo ufw status numbered
```

Question :

Question

Pourquoi est-il nécessaire de conserver une règle autorisant le port 22 lorsque l'administration de la VM est réalisée par SSH ?

3.5 Gestion des utilisateurs et des groupes

Créer un groupe :

```
sudo groupadd cloud-admin
```

Créer un utilisateur :

```
sudo adduser operateur-cloud
```

Ajouter l'utilisateur au groupe :

```
sudo usermod -aG cloud-admin operateur-cloud
```

Vérifier les groupes :

```
groups operateur-cloud
```

Cette manipulation permet d'illustrer le principe du **moindre privilège**.

Étape 4 — Provisionnement automatisé avec Vagrant (25 min)

Cette étape permet d'introduire le concept d' **Infrastructure as Code (IaC)**.

4.1 Préparer le projet Vagrant

Sur le système d'exploitation hôte :

```
mkdir tp-cloud-vagrant
cd tp-cloud-vagrant
```

Initialiser le projet :

```
vagrant init ubuntu/trusty64
```

Le fichier suivant est créé :

```
Vagrantfile
```

4.2 Vérifier que la box est locale

Avant de lancer la VM :

```
vagrant box list
```

Vous devez avoir :

```
ubuntu/trusty64 (virtualbox, ...)
```

Important

Si la box apparaît dans la liste, Vagrant peut créer la VM sans télécharger la box depuis Internet.

4.3 Configuration du Vagrantfile

Remplacer le contenu du fichier `Vagrantfile` par :

```
Vagrant.configure("2") do |config|

config.vm.box = "ubuntu/trusty64"

config.vm.hostname = "vm-tp1-cloud"

config.vm.network "private_network",
ip: "192.168.56.110"

config.vm.provider "virtualbox" do |vb|

vb.name = "vm-tp1-cloud"

vb.memory = 1024

vb.cpus = 1

end

end
```

4.4 Déployer la VM

Exécuter :

```
vagrant up
```

Observer les différentes opérations réalisées automatiquement par Vagrant.
Vérifier :

```
vagrant status
```

4.5 Connexion avec vagrant ssh

Exécuter :

```
vagrant ssh
```

Si tout fonctionne, l'étudiant arrive directement dans Ubuntu :

```
vagrant@vm-tp1-cloud:~$
```

Vérifier :

```
whoami
hostname
hostname -I
```

Observation importante

Avec :

```
vagrant ssh
```

l'étudiant n'a pas besoin de saisir manuellement le mot de passe.
Vagrant utilise automatiquement la configuration SSH et la clé privée associée à la VM.

4.6 Arrêt et suppression de la VM

Quitter la VM :

```
exit
```

Arrêter la VM :

```
vagrant halt
```

Pour supprimer la VM :

```
vagrant destroy -f
```

Attention

La commande `vagrant destroy -f` supprime la machine virtuelle créée par Vagrant.
Elle ne supprime pas la box :

```
ubuntu/trusty64
```

La box reste disponible pour créer ultérieurement une nouvelle VM.

5 Synthèse et comparaison avec GCP

Compléter le tableau suivant.

Concept	**Cloud local**	**Cloud public**
Machine virtuelle	VirtualBox	Compute Engine / EC2 / Azure VM
Réseau virtuel	VirtualBox Private Network	VPC / Virtual Network
Pare-feu	UFW	VPC Firewall / Security Groups
Accès distant	SSH	SSH / Cloud Shell / Console
Gestion des utilisateurs	Linux Users/Groups	IAM
Provisionnement	Vagrant	API / Console / Terraform
Infrastructure	Ordinateur local	Datacenters du fournisseur
Scalabilité	Limitée par le matériel local	Élastique
Facturation	Pas de facturation Cloud	Pay-as-you-go

6 Questions de réflexion

1. Pourquoi une machine virtuelle peut-elle être considérée comme une ressource IaaS ?
2. Quel est le rôle de VirtualBox dans cette infrastructure ?
3. Quel est le rôle de Vagrant ?
4. Quelle est la différence entre une **box Vagrant** et une **machine virtuelle** ?
5. Pourquoi l'enseignant peut-il distribuer une seule box `ubuntu-trusty64.box` à toute la classe ?
6. Quelle commande permet d'ajouter une box locale à Vagrant ?
7. Quelle commande permet de vérifier que la box est disponible localement ?
8. Comment avez-vous obtenu l'adresse IP de la VM ?
9. Quelle commande permet de vérifier que le serveur SSH fonctionne dans Ubuntu Trusty ?
10. Quelle est la différence entre :

    ```
    ssh vagrant@192.168.56.110
    ```

    et :

    ```
    vagrant ssh
    ```

11. Pourquoi faut-il autoriser le port 22 avant d'activer UFW ?
12. En quoi Vagrant illustre-t-il la notion d'*Infrastructure as Code* ?
13. Quelle différence existe-t-il entre une infrastructure locale VirtualBox et une infrastructure Cloud telle que GCP ?

7 Livrables attendus

Chaque étudiant doit remettre un compte-rendu PDF contenant :

— une capture montrant :

```
vagrant box list
```

et la présence de :

```
ubuntu/trusty64
```

— une capture de :

```
vagrant status
```

montrant que la VM est en fonctionnement ;

— une capture de la console Ubuntu montrant :

```
whoami
hostname
hostname -I
```

— une capture montrant les ressources de la VM :

```
lscpu
free -h
df -h
```

— une capture du statut SSH :

```
sudo service ssh status
```

— une capture depuis PowerShell montrant :

```
ping 192.168.56.110
```

et :

```
Test-NetConnection 192.168.56.110 -Port 22
```

— une capture de :

```
sudo ufw status numbered
```

montrant les règles du pare-feu ;

— une capture montrant la création du groupe et de l'utilisateur ;

— une capture du déploiement :

```
vagrant up
```

— une capture de la connexion :

```
vagrant ssh
```

— le tableau de comparaison avec GCP complété ;

— les réponses aux questions de réflexion.

8 Bilan

À l'issue de ce TP, l'étudiant doit être capable de décrire le cycle simplifié suivant :

Box Vagrant → Vagrant → VirtualBox → VM Ubuntu → Réseau → SSH → Administration

Ce fonctionnement constitue une introduction pratique aux concepts qui seront retrouvés ultérieurement dans les infrastructures Cloud publiques :

Virtualisation + Réseau + Sécurité + Automatisation = Infrastructure Cloud